Decision-Bound Verification for Concurrent A2A Agents: Characterizing TOCTOU Failures and Evaluating Semantic Revalidation

Project Team

Draft — November 8, 2026

Abstract

The Agent2Agent (A2A) protocol enables autonomous agents to collaborate through asynchronous, multi-turn interactions. However, concurrent agents may modify shared task state between the time another agent reads information, makes a decision, and executes an action, introducing time-of-check-to-time-of-use (TOCTOU) vulnerabilities. Recent work, A2ABreak, identifies this weakness at the protocol level, but its manifestation in executable multi-agent workflows and the effectiveness of practical interventions remain insufficiently characterized.

We investigate TOCTOU vulnerabilities in A2A systems through two planned contributions. First, we plan to develop a reproducible evaluation framework spanning minimal protocol-level races and realistic multi-agent workflows involving shared resources, delayed decisions, and concurrent updates. Second, we propose a semantic revalidation mechanism in which agents generate explicit decision preconditions, an independent LLM verifier checks their continued validity, and a trusted execution layer enforces the resulting decision using dependency-aware concurrency control and atomic commits.

We plan experiments comparing our approach against unprotected execution, coarse-grained locking, task-wide versioning, and conventional dependency-based revalidation. We will evaluate both security and utility through unsafe-action rates, legitimate-work preservation, and computational overhead.

Our objective is to characterize when stale decisions produce security failures and determine whether semantic verification can preserve decision integrity without unnecessarily restricting parallel agent execution.

1 Introduction

Autonomous agents are increasingly deployed as collaborative systems rather than isolated models. Instead of independently completing tasks, agents delegate subtasks, exchange information, access shared resources, and coordinate across organizational boundaries. Google's Agent2Agent (A2A) protocol standardizes such interactions, providing mechanisms for agent discovery, task delegation, asynchronous execution, and multi-turn communication [1].

However, the ability of agents to operate asynchronously introduces a fundamental security problem: **the state upon which an agent makes a decision may no longer be valid when that decision is acted upon**.

Consider an agent that receives a request to approve a transaction. Before responding, it reads the transaction amount, recipient, and applicable authorization policy. While the agent is evaluating the request, another agent modifies the shared task or its underlying resources. If the

first agent subsequently submits its approval without revalidating the relevant information, the approval may be applied to a transaction materially different from the one originally evaluated.

Figure 1 illustrates how an honest response can become stale even when the task identifier and lifecycle status remain unchanged.

This is a time-of-check-to-time-of-use (TOCTOU) vulnerability. Although TOCTOU is well established in conventional computer systems, its manifestation in LLM-agent workflows introduces additional challenges. Agent decisions can involve extended reasoning, delayed tool execution, human intervention, and multiple independently evolving sources of information. Furthermore, the conditions underlying an agent's decision are frequently expressed in natural language, rather than predefined programmatic predicates.

Lotfi et al. [2] identify **Concurrent Access TOCTOU** as one of eleven vulnerabilities in the A2A protocol. Their specification-level analysis demonstrates that the absence of a standardized task-state versioning mechanism permits stale-state decisions under concurrent access. However, the study primarily establishes the protocol-level vulnerability rather than experimentally characterizing its behavior across executable A2A workflows or evaluating alternative runtime defenses.

A straightforward intervention is to lock shared task state or invalidate decisions whenever the task changes. Although conservative concurrency control can prevent stale-state races, it may unnecessarily interfere with legitimate parallel execution. For instance, a change to an unrelated progress message should not necessarily invalidate an approval based on a transaction's price and recipient.

This motivates a more selective approach: **rather than treating every state change as a security-relevant conflict, determine whether the change invalidates the conditions that justified the original decision**.

We investigate this problem through a two-part study. First, we plan to construct reproducible A2A environments that expose TOCTOU failures under controlled concurrent interactions. Second, we propose a verifier-assisted mechanism that represents agent decisions through explicit acceptance criteria, independently validates those criteria against current task state, and binds successful verification to the eventual execution through atomic enforcement.

Our central research question is:

Can semantic verification of agent decisions prevent TOCTOU vulnerabilities in concurrent A2A workflows while preserving more legitimate parallel execution than conventional task-wide concurrency controls?

1.1 Motivation

Existing concurrency mechanisms provide strong foundations for detecting and preventing stale-state updates. However, long-running agent workflows make the distinction between a **state change** and a **decision-invalidating change** particularly important.

For example, consider an agent approving a purchase conditional on the vendor's authorization status and the presence of a valid warranty. Another agent may modify a vendor document while approval is pending. A minor formatting update should not invalidate the purchase, whereas a change indicating that the vendor is no longer authorized should.

These updates may occur outside the structured fields of the original approval request. Consequently, a fixed set of monitored fields may miss relevant dependencies, while task-wide locking or versioning may unnecessarily reject independent work.

We are interested in whether an LLM-based verifier can identify these semantic dependencies, including dependencies that were not explicitly programmed beforehand, and whether such verification can be integrated with deterministic concurrency enforcement.

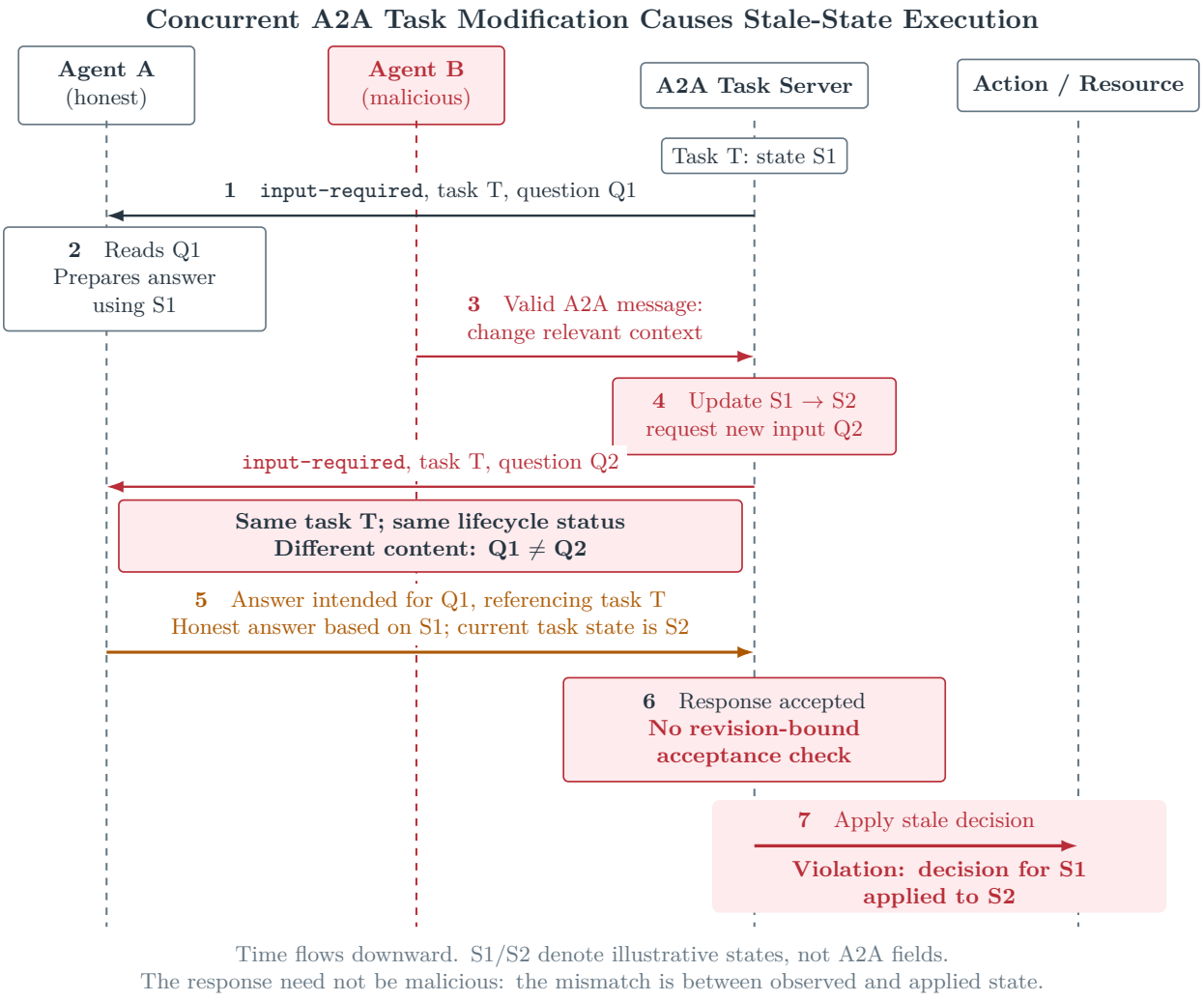


Figure 1: A protocol-compliant concurrent update changes the conditions of an ongoing A2A task after Agent A observes its state. Without revision-bound validation, A's subsequent response may be processed against a different task state from the one it evaluated. S1/S2 are illustrative state labels, not fields supplied by A2A.

The challenge is not merely detecting suspicious modifications. It is preserving the relationship between the **evidence an agent observed**, the **conditions justifying its decision**, and the **state against which that decision is executed**.

1.2 Theory of Change

The security of multi-agent systems depends not only on the behavior of individual agents, but also on the integrity of decisions across interactions with a changing environment. Even well-behaved agents can produce unsafe outcomes when their decisions are executed against stale state.

We hypothesize that expressing agent decisions as independently verifiable conditions can improve the security and efficiency of concurrent agent workflows. By separating semantic decision validation from deterministic state enforcement, a system may identify which concurrent modifications genuinely invalidate a decision without blocking unrelated work.

Our proposed approach operates at three conceptual levels. First, agents expose the acceptance criteria underlying consequential decisions. Second, an independent verifier examines whether those criteria are sufficient and remain satisfied by authoritative task state. Third, a trusted execution mechanism ensures that verification and the corresponding committed action are consistent, rejecting or revalidating operations when relevant state changes.

Figure 2 distinguishes agent computation, independent verification, and atomic enforcement. The commit boundary is proposed application logic, not an existing A2A protocol guarantee.

If successful, this mechanism could provide a reusable foundation for safer A2A interoperability: agents could continue reasoning and collaborating concurrently, while consequential actions remain tied to the conditions that authorize them.

Beyond the proposed intervention, reproducible attack environments could help protocol developers identify where A2A implementations require additional concurrency guarantees and evaluate alternative solutions under comparable conditions.

2 Related Work

2.1 Agent-to-Agent Protocols and Security

The Agent2Agent protocol provides a standardized interface for communication among autonomous agents, including task creation, state tracking, multi-turn exchanges, streaming updates, and asynchronous execution [1]. These capabilities support distributed workflows in which multiple agents exchange information and operate over persistent task contexts.

However, protocol-level interoperability does not necessarily guarantee application-level security. Li et al. [3] introduce A2ASecBench, a protocol-aware evaluation framework covering six attack families across A2A discovery, orchestration, and execution. Their work demonstrates the importance of evaluating security alongside legitimate task utility, but does not specifically benchmark concurrent stale-decision failures.

Lotfi et al. [2] develop A2ABreak, a systematic security analysis of the A2A specification using a formally grounded finite-state-machine framework. Among eleven identified vulnerabilities, their Concurrent Access TOCTOU finding concerns the absence of task-state versioning and conditional update semantics. Their analysis motivates the first part of our work: constructing executable environments to characterize stale-state failures and evaluate their consequences under controlled concurrency.

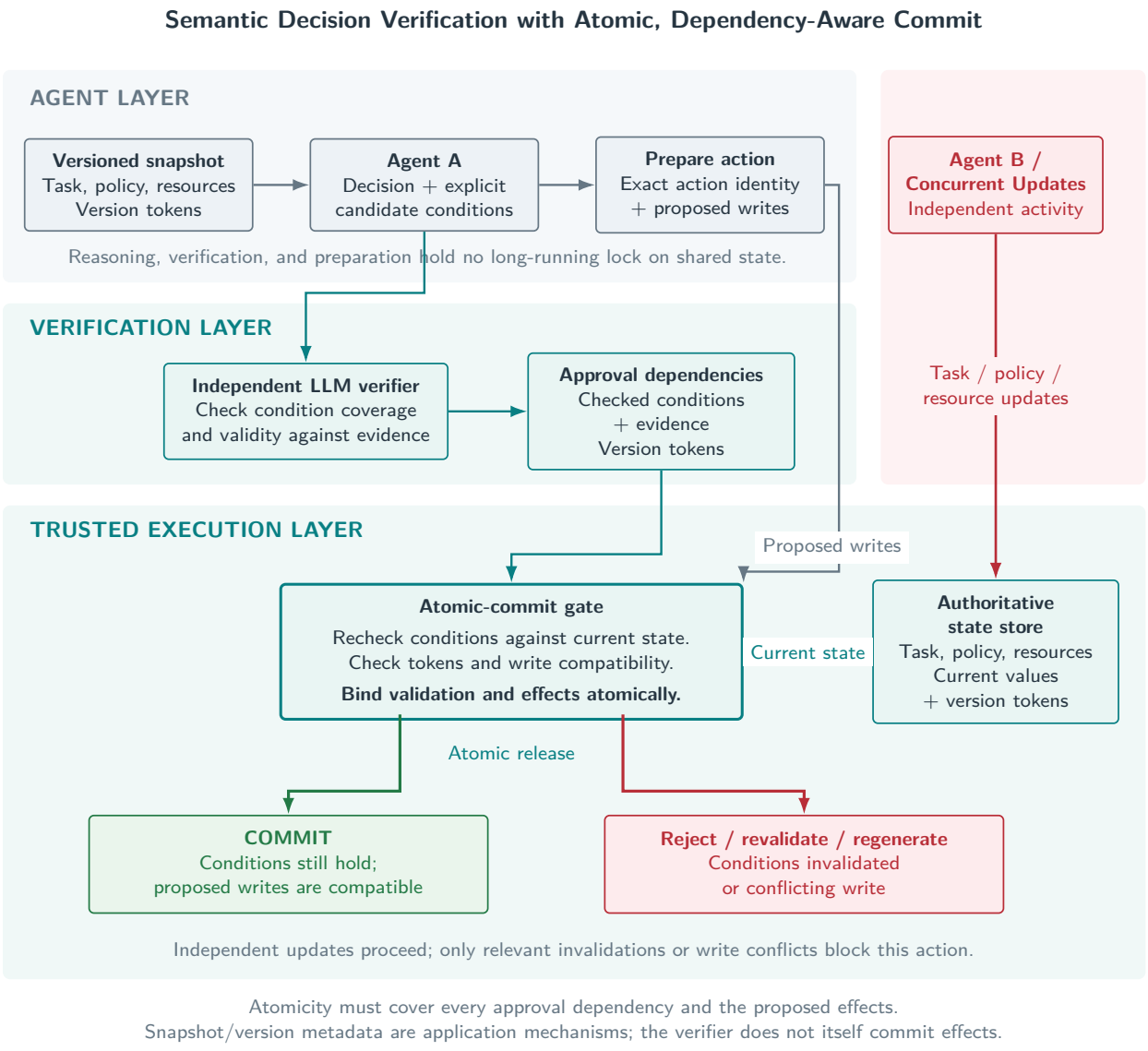


Figure 2: The proposed framework converts agent-generated decision conditions into independently checked approval dependencies. Agents operate on versioned snapshots without holding long-running locks. Before consequential effects are committed, the trusted execution layer atomically validates the relevant conditions against current state, permitting independent updates while rejecting stale or unauthorized actions. This requires atomic enforcement covering the relevant dependencies and proposed effects.

2.2 Concurrency Control in Multi-Agent Systems

Classical concurrency-control mechanisms, including locking and optimistic concurrency control, provide techniques for coordinating access to shared mutable state. However, AI agents introduce long-running operations that may interleave model inference, tool calls, and external interactions, making coarse locking or repeated transaction retries potentially expensive.

Lyu et al. [4] propose CoAgent, a concurrency-control system for multi-agent workflows that incorporates LLM-based judgments of whether conflicting writes invalidate an agent's plan. Rather than relying solely on conventional abort-and-retry behavior, CoAgent supports selective repair of affected operations. This work demonstrates the potential value of semantic reasoning in agent concurrency control.

Peng and Wu [5] investigate stateful governance for concurrent agentic systems, formalizing correctness requirements for actions whose authorization depends on mutable policy state. Their work emphasizes that authorization must remain valid when consequential effects occur, and that concurrency mechanisms should preserve legitimate parallel execution without permitting stale authorization.

These approaches establish that semantic conflict detection and state-dependent authorization are important problems. Our work focuses specifically on adversarial TOCTOU scenarios in A2A communication, with attention to the reliability of automatically generated decision conditions and their use in independent verification.

2.3 Decision-Level Revalidation

Lyu et al. [6] introduce selective revalidation for long-running AI agents, distinguishing ordinary version conflicts from changes that invalidate the justification for an agent's decision. Their proposed runtime records explicit executable preconditions and selectively rechecks affected dependencies before releasing an operation. Atomic transactions or compare-and-set mechanisms are used to bind verified state to execution.

This work is particularly close to ours and provides an important conceptual foundation. However, its prototype relies on developer-specified conditions rather than automatically extracting decision requirements from agent behavior.

We investigate whether agents can generate their own decision conditions and whether independent LLM verification can recover omitted or implicit dependencies. We further situate this problem within A2A-specific concurrent interactions, comparing semantic revalidation against established consistency mechanisms under both malicious and benign modifications.

Our contribution is therefore not a replacement for conventional transaction isolation or atomic enforcement. Rather, we study whether model-based verification can help determine **what must remain true** for an agent's prior decision to stay valid, while trusted concurrency mechanisms guarantee that the relevant checks are enforced at execution.

3 Current Project Scope

An approval such as "proceed with the purchase" leaves its assumptions implicit. A price, vendor authorization, or warranty observed during an initial read can change before the action is accepted. Our goal is to make the approval criteria visible and reviewable, so the application can check whether they still hold.

The initial artifact is an application-level approval system prompt in `prompts/approval_system.txt`, with integration guidance in `docs/approval_workflow.md`. This is a proposed project

convention, not an adopted A2A protocol change. No existing approval-model call site was identified in this checkout; loading the prompt into the approval agent remains an integration task. A separate team workstream owns the test environment under `samples/`.

4 Approval Response

The protocol's in-task authorization workflow already covers human approval: delegated authorization uses `TASK_STATE_AUTH_REQUIRED` with an explanatory status message. Authorization is returned out of band unless an in-band mechanism is negotiated. Its scope and validity are implementation- or extension-defined[1].

The reviewer should explain its recommendation with a concise summary of the evidence and return explicit conditions before acceptance. Conditions may use plain language or pseudocode. An illustrative purchase review is:

```
{
  "decision": "approve",
  "conditions": [
    "total_price_usd <= 600",
    "vendor.authorized == true",
    "warranty.valid == true"
  ],
  "explanation": "The current quote totals 580 USD including all charges; vendor
    authorization and warranty validity are confirmed for this purchase."
}
```

The spending limit and currency are illustrative, not a default policy. Each actual review must use the trusted policy and evidence for the exact proposed action.

The allowed decisions are `approve`, `reject`, and `needs_information`. Approval requires a nonempty condition list covering every applicable policy requirement and evidence supporting each one. A known violation requires rejection. Missing, stale, ambiguous, or conflicting required evidence requires more information. The explanation summarizes the decision-relevant facts rather than requesting private model reasoning.

5 Read, Review, Accept, and Execute

The proposed application workflow is:

1. Read the proposed action and authoritative evidence. Record the action identity, relevant resource versions, and trusted policy.

2. Ask the approval reviewer for a decision, conditions, and explanation. Treat task and vendor content as evidence, not policy.

3. Validate the response and show the explanation and conditions alongside the exact action to the designated acceptance authority.

4. Before acceptance, check all trusted policy requirements against current authoritative data. If execution is delayed, revalidate again before executing the accepted action.

[1] A2A specification, section 7.6: In-Task Authorization, accessed November 8, 2026.

5. Where supported, apply resource-version checks or preconditions atomically with execution. Retain the review and acceptance evidence.

An application's `AgentExecutor.execute` is the intended integration point. A possible authorization handoff is a message with text and structured review data passed to `TaskUpdater.requires_auth`. Application logic must obtain authorization through the agreed channel, check its scope, and perform policy checks. Ordinary follow-up messages do not automatically constitute authorization. The prompt artifact alone does not implement this workflow.

A protocol-level extension would additionally define interoperable action identifiers, condition semantics, authorization validity, and the negotiated return channel. The initial prompt asset does not implement that extension.

6 Enforcement Boundary

A model's `approve` is a recommendation, not user consent or proof that the conditions hold. The application must validate the response, check every trusted requirement independently of the model's list, and bind acceptance to the exact action. Condition strings are descriptive; they must not be evaluated as arbitrary code.

Fresh reads reduce reliance on stale evidence but cannot eliminate a race between a check and an external action. Atomic conditional execution, transactions, or an equivalent mechanism are needed where available. Where unavailable, the remaining race must be recorded as a limitation.

7 Evaluation Plan

Compare a decision-only baseline with the proposed prompt in the team's test environment. Include compliant actions, individual policy violations, missing evidence, contradictory evidence, malformed responses, and task content that attempts to override the policy. Change the price, vendor authorization, warranty, or proposal between the initial read, approval, and execution to test stale decisions and action binding.

Measure response-contract validity, coverage of trusted requirements, unsupported-approval rate, detection of stale approvals, and additional approval latency. Separate prompt adherence from application enforcement: a compliant explanation is not evidence that an action was blocked. Record the model and prompt versions, policy, trial count, evidence versions, mutation timing, and execution outcomes for reproducibility.

8 Implementation Status and Results

Current status: reusable prompt and integration specification; report scaffold. Runtime prompt loading, response validation, trusted policy evaluation, action binding, and acceptance handling are pending.

Results: no approval-model experiment or attack evaluation has been run for this draft. Add measured outcomes and uncertainty after the test environment and runtime integration are available.

References

[1] A2A Project. *Agent2Agent Protocol Specification*, v1.0.0. https://a2a-protocol.org/v1.0.0/specification/

[2] Lotfi, A., Rahman, M. M., Karim, I., & Bertino, E. (2026). *A2ABreak: Systematic Security Analysis of the A2A Protocol*. ACSAC 2026. https://arxiv.org/abs/2609.10871

[3] Li, T., Chu, C., Zheng, Y., Zhang, B., Gong, N. Z., & Xiao, C. (2026). *A2ASecBench: A Protocol-Aware Security Benchmark for Agent-to-Agent Multi-Agent Systems*. ICLR 2026. https://openreview.net/forum?id=LfdFnakqGJ

[4] Lyu, H., Zhang, D., Wu, M., Wei, X., & Chen, H. (2026). *CoAgent: Concurrency Control for Multi-Agent Systems*. https://arxiv.org/abs/2606.15376

[5] Peng, Y., & Wu, X. (2026). *Stateful Governance for Concurrent Agentic Systems*. https://arxiv.org/abs/2608.02764

[6] Lyu, Y., Ren, Y., Lai, R., & Liu, W. (2026). *From Version Conflicts to Decision Conflicts: Selective Revalidation for Long-Running AI Agents*. https://arxiv.org/abs/2609.08015